

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

PLAN DE PRUEBAS

PROYECTO: DieselMaps

Sistema de Geolocalización de Estaciones de Combustible

Fecha: Mayo 2026

Firmado por: Líder de Calidad — Grupo 2

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

1. Introducción

DieselMaps es una aplicación web de geolocalización que permite a conductores y usuarios finales ubicar estaciones de combustible cercanas, consultar precios en tiempo real, comparar tarifas, registrar estaciones favoritas y calcular rutas. La aplicación sigue una arquitectura en tres capas: frontend React + Vite (puerto 5175), backend Spring Boot (puerto 8080) y base de datos MySQL (esquema diesel_maps), con autenticación basada en tokens JWT y tres roles de usuario diferenciados: USER, OPERATOR y ADMIN.

1.1 Objetivo

Garantizar el cumplimiento de los requerimientos funcionales y no funcionales del proyecto DieselMaps mediante la ejecución sistemática de pruebas de integración, sistema, aceptación con usuario final y evaluación de atributos de calidad (carga, desempeño, escalabilidad, disponibilidad y modificabilidad).

1.2 Alcance

El presente documento cubre la validación de los módulos y funcionalidades detectadas en el código fuente del proyecto:

- Módulo de Autenticación: registro y login con JWT (AuthController, AuthService, JwtUtil, JwtFilter)
- Módulo de Estaciones: creación, consulta por proximidad geográfica, detalle, y actualización de precios (StationController, StationService, StationRepository con haversine)
- Módulo de Inventario de Combustible: gestión de stock por tipo de combustible CORRIENTE, EXTRA, DIESEL, GAS (FuelInventoryController, FuelInventoryService)
- Módulo de Compras de Combustible: creación de compra con desglose de ítems, historial personal y por estación (FuelPurchaseController, FuelPurchaseService)
- Módulo Administrativo: gestión global de estaciones por ADMIN (AdminStationController, AdminStationService)
- Módulo de Usuario: favoritos y alertas de precios (UserController, UserService)
- Frontend React: rutas protegidas por rol, integración con Google Maps API, comparador de precios, historial gráfico con Recharts, tienda de combustible
- Seguridad: Spring Security stateless con RBAC (@PreAuthorize), BCrypt, CORS, manejo global de excepciones
- Base de datos: MySQL con Flyway migrations (V1.0.4 a V1.0.7), entidades con relaciones JPA

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

1.3 Definiciones y Abreviaturas

Abreviatura / Término	Descripción
API REST	Interfaz de programación basada en HTTP que expone recursos en el backend Spring Boot
JWT	JSON Web Token – mecanismo de autenticación stateless utilizado en DieselMaps
RBAC	Role-Based Access Control – control de acceso basado en roles (USER, OPERATOR, ADMIN)
DTO	Data Transfer Object – objeto de transferencia de datos entre capas
JPA	Java Persistence API – abstracción para acceso a base de datos relacional
BCrypt	Algoritmo de hashing de contraseñas utilizado para almacenar credenciales
CORS	Cross-Origin Resource Sharing – política de seguridad de recursos entre orígenes
SPA	Single Page Application – arquitectura del frontend React
JMeter	Herramienta Apache para pruebas de carga y rendimiento
SonarQube	Plataforma de inspección de calidad y deuda técnica del código fuente
Haversine	Fórmula matemática usada en StationRepository para cálculo de distancias geográficas
CRUD	Create, Read, Update, Delete – operaciones básicas sobre datos
BD	Base de datos MySQL (esquema: diesel_maps, puerto: 3306)
Frontend	Aplicación React/Vite ejecutada en puerto 5175 (dev)
Backend	Aplicación Spring Boot ejecutada en puerto 8080

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

2. Estrategia General de Pruebas

2.1 Enfoque

Se adoptará una estrategia de pruebas basada en cobertura de requerimientos funcionales reales detectados en el análisis del código fuente. Las pruebas se organizan en los siguientes niveles:

- Pruebas de Integración: validar comunicación frontend–backend–base de datos por módulo funcional.
- Pruebas de Aceptación: validar flujos completos desde la perspectiva del usuario final.
- Pruebas de Sistema: verificar el comportamiento global del sistema como unidad integrada.
- Pruebas de Calidad No Funcional: carga (JMeter), desempeño, escalabilidad, disponibilidad y modificabilidad (SonarQube).

2.2 Ambiente de Pruebas

Componente	Tecnología / Versión	URL / Configuración
Backend	Spring Boot 3.5.5 / Java 17+	http://localhost:8080
Frontend	React 19 + Vite 8 / Node.js 20	http://localhost:5175 (npm run dev)
Base de datos	MySQL Server 8.0	jdbc:mysql://localhost:3306/diesel_maps
Gestor BD	MySQL Workbench 8.0	localhost:3306, usuario: root
Pruebas API	Postman + Scripts Node.js (axios)	Colección DieselMaps API
Mapas	Google Maps API (@react-google-maps/api)	Requiere API Key válida
Carga	Apache JMeter 5.6+	Plan de prueba DieselMaps.jmx
Calidad	SonarQube 10.x Community	http://localhost:9000

2.3 Criterios de Entrada

- Backend Spring Boot levantado y accesible en puerto 8080.
- Base de datos MySQL activa con esquema diesel_maps y al menos un registro de estación.
- Frontend React corriendo en modo desarrollo (npm run dev).
- Variables de entorno y application.properties configuradas correctamente.
- Token JWT válido obtenido mediante autenticación previa para pruebas de endpoints protegidos.
- API Key de Google Maps configurada para pruebas de integración de mapas y rutas.

2.4 Criterios de Salida

- Todos los casos de prueba críticos ejecutados (cobertura $\geq 90\%$ de módulos).
- Defectos críticos (P1) y de alta prioridad (P2) resueltos y verificados.
- Métricas de carga dentro de umbrales aceptables (tiempo de respuesta $\leq 2s$ al 95th percentile).
- Análisis SonarQube con calificación de mantenibilidad no inferior a B.
- Documento firmado por líder de calidad.

2.5 Recursos

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Tipo	Recurso	Rol
Humano	1 Líder QA	Diseño y supervisión de pruebas, firma del plan
Humano	Equipo de desarrollo	Corrección de defectos encontrados
Hardware	Computador con procesador i5+ / 8 GB RAM	Ejecución local de todos los componentes
Software	IntelliJ IDEA / VS Code	Desarrollo y depuración
Software	Postman, Node.js + axios	Pruebas de API y scripts automatizados
Software	Google Chrome DevTools	Inspección de red y frontend
Software	Apache JMeter 5.6+	Pruebas de carga y rendimiento
Software	SonarQube 10.x	Análisis estático de código y deuda técnica

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

3. Plan de Pruebas de Integración

3.1 Estrategia de Integración

Se utilizará la estrategia Bottom-Up con integración incremental por módulo, partiendo desde la capa de persistencia (repositorios JPA → MySQL) hacia la capa de servicios y finalmente hacia los controladores REST. La integración frontend-backend se valida mediante pruebas end-to-end sobre el navegador y mediante Postman para los endpoints.

Flujo de integración validado:

Frontend React (Axios) → API REST Spring Boot (JWT) → Service Layer → Repository JPA → MySQL (diesel_maps)

3.2 Resumen de Casos de Integración

ID	Caso de Uso / Módulo	Módulos Integrados	Resultado Esperado
CPI-001	Registro de usuario (USER/OPERATOR)	AuthController → AuthService → UserRepository → MySQL	Usuario creado, JWT retornado (200 OK)
CPI-002	Autenticación y generación de JWT	AuthController → AuthService → JwtUtil → UserRepository	Token JWT válido con claims de rol
CPI-003	Acceso con token inválido o expirado	JwtFilter → SecurityConfig → GlobalExceptionHandler	HTTP 401 Unauthorized devuelto
CPI-004	Creación de estación (OPERATOR)	StationController → StationService → StationRepository + FuelPriceRepository → MySQL	Estación persistida, HTTP 201, inventario inicializado
CPI-005	Consulta estaciones cercanas (público)	StationController → StationService → StationRepository (haversine query) → MySQL	Lista de estaciones en radio dado, sin JWT requerido
CPI-006	Detalle de estación (autenticado)	StationController → StationService → StationRepository → MySQL	DTO completo con precios e inventario
CPI-007	Actualización de precio (OPERATOR/ADMIN)	StationController → StationService → FuelPriceRepository + PriceHistoryRepository → AlertRepository	Precio actualizado, historial registrado, alertas generadas
CPI-008	Consulta inventario por estación	FuelInventoryController → FuelInventoryService → FuelInventoryRepository → MySQL	Lista de inventario por

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

ID	Caso de Uso / Módulo	Módulos Integrados	Resultado Esperado
			tipo de combustible
CPI-009	Ajuste de stock (OPERATOR/ADMIN)	FuelInventoryController → FuelInventoryService → FuelInventory.adjustStock → FuelInventoryHistoryRepository	Stock actualizado, historial registrado
CPI-010	Creación de compra de combustible	FuelPurchaseController → FuelPurchaseService → FuelInventory.completeImmediatePurchase → FuelPurchaseRepository	Compra creada (201), stock descontado en BD
CPI-011	Historial de compras del usuario	FuelPurchaseController → FuelPurchaseService → FuelPurchaseRepository → MySQL	Lista de compras del usuario autenticado
CPI-012	Gestión administrativa de estaciones (ADMIN)	AdminStationController → AdminStationService → StationRepository → MySQL	CRUD completo de estaciones desde panel admin
CPI-013	Toggle de favoritos	UserController → UserService → UserFavoriteRepository → MySQL	Favorito creado o eliminado según estado previo
CPI-014	Consulta y marcado de alertas	UserController → UserService → AlertRepository → MySQL	Lista de alertas del usuario; alerta marcada como leída
CPI-015	Integración Google Maps (rutas)	Frontend MapView/Routes → Google Maps API (externa)	Ruta calculada y visualizada en mapa; fallido si API Key inválida
CPI-016	Historial de precios de estación	StationController → StationService → PriceHistoryRepository → Frontend (Recharts)	Gráfica de historial renderizada correctamente
CPI-017	Control de acceso por rol (403)	JwtFilter → @PreAuthorize → GlobalExceptionHandler	HTTP 403 Forbidden para rol insuficiente
CPI-018	Manejo de base de datos no disponible	StationController → StationService → MySQL (CAÍDA)	HTTP 500 o error de conexión retornado; no excepción descontrolada

3.3 Casos de Prueba Detallados

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

CASO DE INTEGRACIÓN CPI-00X — REGISTRO DE USUARIO		
ID		
Caso de Uso		
Objetivo		
Precondiciones		
Módulos Integrados		
DETALLE DEL FLUJO DE PRUEBA		
No.	Acción / Entrada	Resultado Esperado
Resultado Esperado:		
Criterio de Aceptación:		
Estado: APROBADO / FALLIDO		Responsable: Jonathan cobos y Camilo Barbosa

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

4. Plan de Pruebas de Aceptación con Usuario Final

4.1 Estrategia

Las pruebas de aceptación validan que el sistema satisface las necesidades del usuario final desde una perspectiva funcional y de experiencia de uso. Se diseñan escenarios basados en los flujos principales detectados en el frontend React (App.jsx, páginas y contexto de autenticación) y se ejecutan directamente en el navegador sin conocimiento de la implementación técnica.

4.2 Resumen Prueba de Aceptación

4.3 Casos de Prueba de Aceptación

CASO DE ACEPTACIÓN CPA-00x		
ID		
Caso de Uso		
Objetivo		
Precondiciones		
Módulos Integrados		
DETALLE DEL FLUJO DE PRUEBA		
Resultado Esperado General:		
Criterio de Aceptación:		
Estado: APROBADO / FALLIDO	Responsable:	

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

5. Plan de Pruebas de Sistema

5.1 Estrategia

Las pruebas de sistema evalúan el comportamiento del sistema DieselMaps como un todo integrado, verificando que todos los módulos interactúen correctamente bajo condiciones reales de operación. Se ejecutan sobre el entorno local completo con todos los servicios activos simultáneamente.

5.2 Casos de Prueba de Sistema

ID	Escenario del Sistema	Entrada	Resultado Esperado	Estado
CPS-001	Flujo completo usuario nuevo: registro → login → mapa → favorito → alerta	Datos de registro válidos, coordenadas Bogotá	Usuario creado, mapa visible, favorito guardado, alerta generada al cambiar precio	APROBADO
CPS-002	Flujo operador: login → crear estación → actualizar precio → ver historial	Credenciales operador, datos estación completos	Estación en mapa, precio actualizado, historial en BD, alerta a usuarios favoritos	APROBADO
CPS-003	Flujo admin: login → panel /admin/stations → desactivar → reactivar estación	Credenciales admin	Estación desaparece y reaparece del mapa de usuarios; estado en BD consistente	APROBADO
CPS-004	Flujo compra: usuario ve stock → compra → stock decrementado → historial actualizado	30L de CORRIENTE en estación con 50L disponibles	Stock: 50 → 20L; compra en historial; total calculado correctamente	APROBADO
CPS-005	Control de sesión: token JWT expirado → redirección automática a login	Token expirado (modificar exp en BD o esperar 24h)	Interceptor 401 detectado; localStorage limpiado; redirección a /login	APROBADO
CPS-006	Acceso a rutas protegidas sin autenticación	Request directo a /map sin token	ProtectedRoute redirige a /login; API retorna 401	APROBADO
CPS-007	Consistencia de datos: actualización de precio refleja en historial y alertas	PATCH /api/stations/:id/prices con nuevo precio	PriceHistory creado, FuelPrice.prevPriceCop actualizado, Alert generada	APROBADO
CPS-008	Manejo de error global: endpoint inexistente	GET /api/nonexistent	HTTP 404 o 400; no excepción no controlada expuesta al cliente	PARCIAL
CPS-009	Estabilidad bajo operaciones simultáneas: 5 usuarios consultando nearby simultáneamente	5 peticiones concurrentes GET /api/stations/nearby	Todas responden correctamente; sin deadlocks en MySQL; tiempos < 1s	APROBADO

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

ID	Escenario del Sistema	Entrada	Resultado Esperado	Estado
CPS-010	Validación de campos obligatorios en todos los formularios	Enviar formularios con campos vacíos	HTTP 400 con mapa de errores por campo; frontend muestra mensajes	PARCIAL — Frontend no muestra todos los errores de la API

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

6. Pruebas de Atributos de Calidad

6.1 Pruebas de Carga (Apache JMeter)

6.1.1 Configuración del Plan de Prueba JMeter

Se diseñan tres escenarios de carga progresivos sobre los endpoints críticos del backend DieselMaps. La herramienta recomendada es Apache JMeter 5.6+ con el plugin JP@GC Standard Set para métricas avanzadas.

Parámetro JMeter	Escenario Base	Escenario Medio	Escenario Pico
Usuarios concurrentes (Threads)	10	50	100
Ramp-up period (segundos)	30	60	90
Duración de prueba (segundos)	120	300	600
Loop count	Forever (durante duración)	Forever	Forever
Think time entre requests (ms)	500–1000	300–700	100–500
Errores aceptables (%)	< 1%	< 2%	< 5%

6.1.2 Endpoints a Probar

Endpoint	Método	Autenticación	Prioridad	Justificación
POST /api/auth/login	POST	No	ALTA	Alta concurrencia en horarios pico; BCrypt.matches es costoso computacionalmente
GET /api/stations/{id}	GET	JWT	ALTA	Consulta frecuente al ver detalle; relación lazy FuelPrice
POST /api/fuel/compras	POST	JWT	ALTA	Transacción con bloqueo optimista de inventario; riesgo de race condition
GET /api/fuel/inventory/station/{id}	GET	JWT	MEDIA	Consulta de stock antes de compra
PATCH /api/stations/{id}/precio	PATCH	JWT OPERATOR	MEDIA	Actualización con escritura en PriceHistory y AlertRepository
GET /api/admin/stations	GET	JWT ADMIN	BAJA	Uso administrativo; no en ruta crítica de usuario

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

6.2 Evaluación de Desempeño

El análisis de desempeño identifica los componentes del sistema DieselMaps con mayor impacto en tiempos de respuesta y potenciales cuellos de botella basados en el código fuente analizado.

Componente Analizado	Riesgo de Desempeño	Causa Técnica Detectada	Recomendación
StationRepository.findNearby (haversine)	ALTO	Fórmula haversine en SQL sin índice espacial en lat/lng; full table scan posible	Agregar índice compuesto (latitude, longitude); considerar índice espacial SPATIAL INDEX
BCryptPasswordEncoder.matches en login	MEDIO	BCrypt con factor de trabajo 10 por defecto; lento intencionalmente (seguridad)	Configurar strength=10-12; usar caché de sesión o rate limiting en /api/auth/login
FuelPurchaseService.createPurchase	MEDIO	Transacción con múltiples escrituras en 3 tablas + lógica de stock	@Transactional correctamente; agregar timeout de transacción; monitorear deadlocks
StationService.updatePrice con alertas	MEDIO	Loop sobre todos los usuarios con favorito en AlertRepository.save()	Usar batch insert para alertas; considerar procesamiento asíncrono (@Async o cola)
GET /api/admin/stations (findAll)	MEDIO	Retorna TODAS las estaciones sin paginación	Implementar Pageable; agregar índice en updated_at para ordenamiento
Carga inicial AuthContext en React	BAJO	localStorage parsing en cada montaje de componente	Memoización con useMemo ya implementada; adecuado
Recharts PriceHistoryChart	BAJO	Renderizado de gráfica con datos históricos grandes	Implementar paginación o ventana de tiempo en getPriceHistory

6.3 Escalabilidad

La arquitectura monolítica de DieselMaps (Spring Boot + MySQL en servidor único) presenta las siguientes características de escalabilidad:

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2 Ciclo: 2

Dimensión	Estado Actual	Escalabilidad
Usuarios concurrentes	Sesión stateless (JWT)	Aumentar maximum-pool-size; load balancer con múltiples instancias
Volumen de datos de estaciones	Sin límite lógico implementado	Implementar Pageable; particionar tabla si > 100K registros
Historial de precios	Tabla price_history sin TTL	Agregar partición por fecha; archivado automático > 1 año
Historial de inventario	fuel_inventory_history con índice en station_id+recorded_at	Implementar purge job periódico
Carga geográfica simultánea	Haversine en BD	Implementar Redis con TTL de 5 minutos para resultados nearby por zona
Google Maps API	Integración directa frontend	Implementar rotación de API Keys; monitorear cuota
Frontend (SPA React)	Vite SPA estática	Servir build estático desde Nginx/CDN; habilitar gzip

6.4 Disponibilidad

La disponibilidad del sistema depende de la corrección de varios puntos de fallo identificados. A continuación se documentan los escenarios de disponibilidad y los comportamientos observados:

Escenario	Comportamiento Actual	Comportamiento Esperado	Estado
MySQL no disponible	HTTP 400 por GlobalExceptionHandler captura genérica	HTTP 503 Service Unavailable con mensaje claro	DEFICIENTE
Backend caído	Frontend muestra error de red genérico de Axios	opción de reintentar	DEFICIENTE
Google Maps API no disponible	Mapa no carga; sin mensaje al usuario	Mensaje explicativo; funcionalidad básica sin mapa	DEFICIENTE
Token JWT expirado	Interceptor Axios detecta 401; redirige a /login	Comportamiento correcto implementado	ADECUADO
Request a endpoint no existente	Spring Boot retorna 404 o WhitelabelError	404 manejado; no exponer stack trace	ADECUADO
Reinicio del backend	Sesiones JWT persisten (stateless)	Usuarios autenticados no pierden sesión	ADECUADO

6.4.1 Pruebas de Disponibilidad con JMeter

	Plan de Pruebas		
Universidad Piloto de Colombia	PROYECTO: DieselMaps		Equipo: 2
			Ciclo: 2

- Soak Test (prueba de resistencia): 30 usuarios durante 60 minutos continuos para detectar memory leaks y degradación progresiva.
- Spike Test: incremento repentino de 5 a 100 usuarios en 10 segundos para simular pico de tráfico; medir tiempo de recuperación.
- Failover Test: detener MySQL durante la prueba de carga y medir cuántos requests fallan y qué respuesta reciben.
- Recovery Test: reiniciar MySQL después de 30 segundos de indisponibilidad; medir tiempo de auto-recuperación del pool HikariCP.

	Plan de Pruebas	
Universidad Piloto de Colombia	PROYECTO: DieselMaps	Equipo: 2
		Ciclo: 2

7. Asignación de Responsabilidades

Caso de Prueba	Tipo	Responsable Ejecución	Rol
CPI-001, CPI-002	Integración	Camilo Barbosa Lis	Líder de Pruebas de Integración (LA)
CPI-003, CPI-017	Integración	Camilo Barbosa Lis	Tester de Seguridad
CPI-004, CPI-005	Integración	Jonathan David Cobos Prieto	Tester Backend Estaciones
CPI-006, CPI-007	Integración	Camilo Barbosa Lis	Tester Backend Precios
CPI-010, CPI-011	Integración	Jonathan David Cobos Prieto	Tester Backend Compras
CPI-015, CPI-016	Integración	Jonathan David Cobos Prieto	Tester Integración Externa (Maps)
CPA-001 a CPA-006	Aceptación	Camilo Barbosa Lis	Testers de Aceptación
CPS-001 a CPS-010	Sistema	Jonathan David Cobos Prieto	Coordinador de Pruebas de Sistema
Pruebas JMeter	Carga	Camilo Barbosa Lis	QA No Funcional
Análisis SonarQube	Modificabilidad	Camilo Barbosa Lis	Análisis conjunto